

RTL-SDR Receiver

Chain K — Hardware & Systems Foundations · Full written lesson

Software-defined radio turns "radio" from a sealed black box into something you can actually see: raw RF comes in over USB, and software does the demodulation a dedicated chip would normally hide from you. A receiver has no transmit stage, so there's no license or legal risk.

Lesson 1 — IQ Sampling: Why Software Does What a Chip Used to Hide

A traditional radio hides demodulation behind dedicated analog or digital circuitry built for exactly one signal type. An SDR dongle instead hands you raw samples and lets software do that math — which means the same \$30 dongle can decode FM broadcast, aircraft transponders, or weather satellites, just by running different math on the same kind of data.

- Each **IQ sample** is a complex number — an In-phase (I) and Quadrature (Q) component together — capturing both the signal's instantaneous magnitude (amplitude) and phase at that moment, not just a single real-valued number the way a simple amplitude reading would.
- The dongle samples a whole chunk of spectrum around whatever frequency it's tuned to (up to a few MHz wide) all at once, as a continuous stream of these IQ samples.
- **Demodulation is just math on that stream** once you have it: AM demodulation reads the magnitude, FM demodulation reads how fast the phase is changing — genuinely different algorithms applied to the exact same kind of raw data.

The trap: buying a generic "USB TV tuner" instead of a dongle explicitly sold as RTL-SDR. Many look identical in photos but use a different tuner chip entirely — worse performance, and sometimes not compatible with SDR software at all. Buy one explicitly branded/sold as RTL-SDR (the RTL2832U + R820T2 chipset combination is the de facto standard).

Worked example: AM demodulation and FM demodulation are genuinely different algorithms — magnitude versus phase-rate-of-change — run on the exact same kind of IQ sample stream.

Lesson 2 — Demodulating FM Broadcast: The Simplest Starting Point

FM broadcast radio is the easiest first real signal: strong, wideband, and well within any RTL-SDR's covered spectrum — a good place to confirm the whole signal chain (antenna → dongle → GQRX → audio) actually works before decoding anything harder.

- **GQRX's waterfall display** plots frequency (horizontal) against time (vertical), with color showing signal strength — a visual map of everything currently on the air, before you've demodulated anything.
- Select **WFM (mono or stereo)** as the demodulation mode for broadcast FM, tune to a strong local station's frequency, and adjust **squelch** (a threshold that mutes audio below a certain signal strength) so weak noise doesn't drown out a real signal.

- The same 433MHz ISM band many simple wireless devices (weather stations, sensors, remotes) use is decodable with `rtl_433` — worth exploring once FM broadcast is working reliably.

The trap: USB port RF noise swamping weak signals. A dongle plugged directly into a laptop's USB port sits right next to a noisy source of RF interference — the computer itself. A basic USB extension cable, moving the dongle a few feet away, fixes this far more often than any software setting does.

Worked example: a station that's inaudible (or drowned in static) plugged directly into a laptop often comes in clean once the dongle is moved a few feet away on an extension cable — same antenna, same software, same frequency, only the USB noise source changed.

Lesson 3 — Decoding Real Aircraft ADS-B Transponder Messages

ADS-B ("Automatic Dependent Surveillance-Broadcast") is the modern replacement for a lot of radar-based tracking: aircraft broadcast their own position, altitude, and identity on 1090MHz, and `dump1090` turns those raw broadcasts into a live map of nearby traffic.

- Every ADS-B **Extended Squitter** message's first byte encodes a 5-bit **Downlink Format (DF)** in its top bits — DF 17 specifically means "this is an ADS-B Extended Squitter," the message type this project decodes.
- The next 3 bytes (24 bits, byte-aligned right after the DF byte) are the aircraft's **ICAO address** — a fixed identifier for that specific airframe. It stays constant across every message from that plane, even as position, altitude, and velocity fields change message to message.
- Later bytes carry position (using Compact Position Reporting, which needs two messages to resolve an exact location), altitude, velocity, or callsign, depending on the message's type code — and the whole message ends in a CRC for integrity.

The trap: a mismatched antenna for the wrong band. A generic FM whip receives 1090MHz far worse than a dedicated ADS-B antenna tuned for that specific frequency.

Worked example: the raw message `8D4840D6202CC371C32CE0576098` decodes by hand to DF 17 (top 5 bits of byte 0: `10001101` → `10001` = 17) and ICAO address `4840D6` (the next 3 bytes) — exactly what `dump1090` 's own output reports for that message.

Lesson 4 — Receiving NOAA Weather Satellites, and Why Antenna Matching Changes Everything

NOAA's polar-orbiting weather satellites transmit APT (Automatic Picture Transmission) images at ~137MHz as they pass overhead — a roughly 10-15 minute window where the same \$30 dongle can pull down an actual, real weather photo of your region.

- A pass is recorded as an audio-like signal, then decoded (with `noaa-apt` or `WXtoIMG`) into a real image — line by line, matching how the satellite actually scans the Earth below it.
- **Antenna matching matters more here than almost anywhere else in this project.** A dedicated 137MHz antenna (a QFH — quadrifilar helix — or even a simple V-dipole) dramatically outperforms a generic whip, partly because it's tuned for the exact frequency and partly because a QFH's circular polarization matches the satellite's own rotating polarization as it passes overhead.

The trap: a whip antenna with no ground plane underperforming badly indoors. This is the single biggest lever in the entire project — the same dongle and software can go from unusable noise to a clean image purely from an antenna and location change, with zero software troubleshooting involved.

Worked example: the exact same RTL-SDR dongle and decoding software can produce a garbled, unusable image with a mismatched indoor whip antenna, and a clean two-channel (visible + infrared) image with a matched outdoor antenna during the same satellite pass.

Reflection

What surprised you most about how much of "radio" turns out to be software, once you have raw IQ samples to work with? Why does a receive-only project like this carry no license requirement, while transmitting on the same frequencies would?